Overview

This installation guide is suitable for machines running MacOS, Windows, and Ubuntu. The instructions for each are included below.

Please note, if you are working on a FIRGA machine, please follow all instructions under the `FIRGA machine` heading.

If you are working on your own machine:

- If your local machine is running Ubuntu, please follow all instructions under the `Ubuntu local machine` heading.
- If your local machine is running MacOS, please follow all instructions under the `MacOS local machine` heading.
- If your local machine is running Windows, please follow all instructions under the `Windows local machine` heading.

FIRGA machine

- Boot in Ubuntu.
- In Visual Studio Code, install the following extensions
  - `C/C++ Extension Pack`
- From SUNLearn, download `emdw_de424.tgz`
- Confirm that it is downloaded into `Downloads`. Navigate to your home directory in the terminal, and extract the tarball using `tar -xvzf Downloads/emdw_de424.tgz` such that the following three directories exist:
  - `~/devel/gLinear`
  - `~/devel/prlite`
  - `~/devel/emdw`
- Add `~/bin` to your search path by including the following line at the bottom of your `~/.bashrc` file:

  ```
  export PATH=$HOME/bin:$PATH
  ```

  Your bin directory will now automatically be included in the search path for all future terminal windows you open. An existing terminal window can be updated by doing a:

  ```
  source ~/.bashrc
  ```

- Symlink the necessary libraries from your `~/bin` directory

  ```
  cd ~/bin
  ln -sf $HOME/devel/gLinear/build/$(uname -s)-Release-$(uname -m)/libgLinear.so
  ln -sf ~/devel/prlite/build/src/libprlite.so
  ln -sf ~/devel/emdw/build/src/libemdw.so
  ```

  Initially those symlinks will be invalid (coloured red), but don't worry. They will get fixed later.

- Compile the `gLinear` library

  ```
  cd ~/devel/gLinear
  mkdir -p build
  ./cmake/configure -g Release
  cd build/Linux-Release-x86_64
  make -j3
  ```

  There might be some warnings, but if you see the lines

  ```
  Built target gLinear
  Built target testnew
  ```

  everything should be fine.

- Compile the `prlite` library

  ```
  cd ~/devel/prlite
  mkdir -p build
  cd build
  cmake ../
  make prlite -j4
  ```

  If you see the line

  ```
  Built target prlite
  ```

  everything should be fine.

- Compile the `emdw` library

```
cd ~/devel/emdw
mkdir -p build
cd build
cmake ../
make emdw -j4
```

If you see the line

```
Built target emdw
```

everything should be fine.

- In Visual Studio Code, click on `Open Folder`
- Navigate to `/home/<username>/devel/emdw/`, click `Open` and click `Yes, I trust the authors`.
- Select the compiler kit to be the largest `GCC` if there are multiple `GCC`'s
- Test if the installation was successful by opening a terminal in Visual Studio Code and typing

  ```
  cd build; cmake ../; make -j7 example; src/bin/example; cd ..
  ```

- If you want to create a new target `week1.cc`
  - Copy `devel/emdw/src/bin/example.cc` to `devel/emdw/src/bin/week1.cc`
  - Open `devel/emdw/src/bin/CMakeLists.txt` and duplicate the bottom two lines
  - Edit these new lines by replacing `example` with `week1`
  - Edit `week1.cc` for your purpose
  - Execute the target as follows

    ```
    cd build; cmake ../; make -j7 week1; src/bin/week1; cd ..
    ```

- Your setup and work should be available on all `FIRGA` machines, but remember to regularly back up your work in a different location.

Ubuntu local machine

- Download and install Visual Studio Code (https://code.visualstudio.com/download)
- In Visual Studio Code, install the following extensions
  - `C/C++ Extension Pack`
- Navigate to the terminal
- Install some base packages and libraries

```
sudo apt-get install build-essential cmake g++ clang liblapack-dev libblas-dev

sudo apt-get install libexpat1-dev zlib1g-dev libboost-all-dev libssl-dev graphviz

sudo apt-get install texlive-latex-base texlive-latex-extra texlive-pictures
```

- From SUNLearn, download `emdw_de424.tgz`
- Confirm that it is downloaded into `Downloads`. Navigate to your home directory in the terminal, and extract the tarball using `tar -xvzf Downloads/emdw_de424.tgz` such that the following three directories exist:
  - `~/devel/gLinear`
  - `~/devel/prlite`
  - `~/devel/emdw`
- Add `~/bin` to your search path by including the following line at the bottom of your `~/.bashrc` file:

```
export PATH=$HOME/bin:$PATH
```

  Your bin directory will now automatically be included in the search path for all future terminal windows you open. An existing terminal window can be updated by running:

```
source ~/.bashrc
```

- Symlink the necessary libraries from your `~/bin` directory

```
cd ~/bin
ln -sf $HOME/devel/gLinear/build/$(uname -s)-Release-$(uname -m)/libgLinear.so
ln -sf ~/devel/prlite/build/src/libprlite.so
ln -sf ~/devel/emdw/build/src/libemdw.so
```

  Initially those symlinks will be invalid (coloured red), but don't worry. They will get fixed later.

- Compile the `gLinear` library

```
cd ~/devel/gLinear
mkdir -p build
./cmake/configure -g Release
cd build/Linux-Release-x86_64
make -j3
```

  There might be some warnings, but if you see the lines

```
Built target gLinear
Built target testnew
```

  everything should be fine.

- Compile the `prlite` library

```
cd ~/devel/prlite
mkdir -p build
```

```
cd build
cmake ../
make prlite -j4
```

If you see the line

```
Built target prlite
```

everything should be fine.

- Compile the `emdw` library

```
cd ~/devel/emdw
mkdir -p build
cd build
cmake ../
make emdw -j4
```

If you see the line

```
Built target emdw
```

everything should be fine.

- In Visual Studio Code, click on `Open Folder`
- Navigate to `/home/<username>/devel/emdw/`, click `Open` and click `Yes, I trust the authors`.
- Select the compiler kit to be the largest `GCC` if there are multiple `GCC`'s
- Test if the installation was successful by opening a terminal in Visual Studio Code and typing

```
cd build; cmake ../; make -j7 example; src/bin/example; cd ..
```

- If you want to create a new target `week1.cc`
  - Copy `devel/emdw/src/bin/example.cc` to `devel/emdw/src/bin/week1.cc`
  - Open `devel/emdw/src/bin/CMakeLists.txt` and duplicate the bottom two lines
  - Edit these new lines by replacing `example` with `week1`
  - Edit `week1.cc` for your purpose
  - Execute the target as follows

```
cd build; cmake ../; make -j7 week1; src/bin/week1; cd ..
```

MacOS local machine

- Download and install Visual Studio Code (https://code.visualstudio.com/download)
- In Visual Studio Code, install the `C/C++ Extension Pack` extension
  - This will likely prompt you to install Apple's Command Line Tools. If it doesn't, navigate to the terminal and run `xcode-select --install` in the terminal and follow the install process.
  - Verify that clang is installed by running `clang --version` in the terminal; it should return something similar to the following:

    ```
    Apple clang version 16.0.0 (clang-1600.0.26.6)
    Target: arm64-apple-darwin24.3.0
    Thread model: posix
    InstalledDir: /Library/Developer/CommandLineTools/usr/bin
    ```
- Download and install MacPorts (https://www.macports.org/install.php)
  - After install, open a new terminal window or run `exec $SHELL -l` and then run `sudo port -v selfupdate` to make sure you are using the latest version.
- Install some base packages and libraries (this is the longest step):

  ```
  sudo port install -N OpenBLAS +lapack

  sudo port install -N boost graphviz

  sudo port install -N texlive-basic texlive-latex-extra texlive-pictures
  ```

  (OpenBLAS completes the other dependencies such as `cmake expat openssl`)
- From SUNLearn, download `emdw_de424.tgz`
- Confirm that it is downloaded into `Downloads`.
- Extract the tarball (if not done automatically) by double clicking it in Finder. Move the `bin` and `devel` folders to your home directory (by dragging and dropping them) such that the following four directories exist:
  - `~/bin`
  - `~/devel/gLinear`
  - `~/devel/prlite`
  - `~/devel/emdw`

  If you cant find your home folder in Finder, in the menu bar click on `Finder` -> `Settings` then the `Sidebar` tab, and click the checkbox with the house icon and your username. Your home directory should now show up in the Finder sidebar.
- Add `~/bin` to your search path by including the following line at the bottom of your `~/.zprofile` file:

  ```
  export PATH="$HOME/bin:$PATH"
  ```

  The `.zprofile` is in the home directory as a hidden file, which can be shown with the keyboard shortcut `shift command .` (the period key). If the file doesn't exist, run the command

  ```
  touch ~/.zprofile
  ```

  in the terminal.

  Your bin directory will now automatically be included in the search path for all future terminal windows you open.

  You will also need to add the following to `~/.zprofile` to link the dependencies:

  ```
  export LIBRARY_PATH=$LIBRARY_PATH:/opt/local/libexec/openssl3/lib/
  ```

An existing terminal window can be updated by running:

```
source ~/.zprofile
```

- Symlink the necessary libraries from your `~/bin` directory

```
cd ~/bin
ln -sf $HOME/devel/gLinear/build/$(uname -s)-Release-$(uname -m)/libgLinear.dylib
ln -sf ~/devel/prlite/build/src/libprlite.dylib
ln -sf ~/devel/emdw/build/src/libemdw.dylib
```

Initially those symlinks might be invalid (coloured red), but don't worry. They will get fixed later.

- Compile the `gLinear` library

```
cd ~/devel/gLinear
mkdir -p build
./cmake/configure -g Release
```

You should get an output similar to this (Silicon chip):

```
## To continue the build process run the following command(s):

cd build/Darwin-Release-arm64
make -j3
sudo make install
```

or this (Intel chip):

```
## To continue the build process run the following command(s):

cd build/Darwin-Release-x86_64
make -j3
make install
```

Copy the first two lines, e.g.:

```
cd build/Darwin-Release-arm64
make -j3
```

and run them in the terminal.

There might be some warnings, but if you see the lines

```
[ 77%] Linking CXX shared library libgLinear.dylib
[ 77%] Built target gLinear
[100%] Linking CXX executable testnew
[100%] Built target testnew
```

everything should be fine.

- Compile the `prlite` library

```
cd ~/devel/prlite
mkdir -p build
cd build
cmake ../
make prlite -j4
```

If you see the line

```
[  5%] Linking CXX shared library libprlite.dylib
[100%] Built target prlite
```

everything should be fine.

- Compile the `emdw` library

  ```
  cd ~/devel/emdw
  mkdir -p build
  cd build
  cmake ../
  make emdw -j4
  ```

  If you see the line

  ```
  [100%] Linking CXX shared library libemdw.dylib
  [100%] Built target emdw
  ```

  everything should be fine.

- In Visual Studio Code, click on `Open Folder`
- Navigate to `~/devel/emdw/`, click `Open` and click `Yes, I trust the authors`.
- Select the compiler kit to be the largest `gcc` if there are multiple `gcc`'s
- Test if the installation was successful by opening a terminal in Visual Studio Code and typing

  ```
  cd build; cmake ../; make -j7 example; src/bin/example; cd ..
  ```

- If you want to create a new target `week1.cc`
  - Copy `devel/emdw/src/bin/example.cc` to `devel/emdw/src/bin/week1.cc`
  - Open `devel/emdw/src/bin/CMakeLists.txt` and duplicate the bottom two lines
  - Edit these new lines by replacing `example` with `week1`
  - Edit `week1.cc` for your purpose
  - Execute the target as follows

    ```
    cd build; cmake ../; make -j7 week1; src/bin/week1; cd ..
    ```

Boost Warning

If, when making `emdw`, the following warning is printed:

```
CMake Warning (dev) at CMakeLists.txt:19 (find_package):
  Policy CMP0167 is not set: The FindBoost module is removed.  Run "cmake
  --help-policy CMP0167" for policy details.  Use the cmake_policy command to
  set the policy and suppress this warning.

This warning is for project developers.  Use -Wno-dev to suppress it.
```

This is silenced by adding

```
cmake_policy(SET CMP0167 NEW)
```

to `emdw/CMakeLists.txt` at line 19 just below

```
# Required boost support
```

Windows local machine

WSL allows running Linux distributions on Windows without a virtual machine or dual boot. It provides access to Linux commandline tools, utilities, and apps. Also, its useful for development, scripting, and system administration on Windows. To install WSL:

- Open powershell as administrator and run

  ```
  wsl --install
  ```

- Create a unix username and password in the terminal if prompted (remember this password as it will allow sudo access)
- Download Ubuntu from the Microsoft Store
- Launch Ubuntu
- You might only be prompted for a username and password once you open the Ubuntu terminal
- Verify the installation in powershell:

  ```
  wsl --list --verbose
  ```

  This should return something like

  ```
    NAME      STATE           VERSION
  * Ubuntu    Running         2
  ```

- Download and install Visual Studio Code (https://code.visualstudio.com/download)
- You can directly open Visual Studio from the terminal setting the current directory as the workspace by running in the Ubuntu terminal

  ```
  code .
  ```

- In Visual Studio Code, install the following extensions
  - `C/C++ Extension Pack`
- Navigate to the terminal
- Install some base packages and libraries

  ```
  sudo apt-get install build-essential cmake g++ clang liblapack-dev libblas-dev

  sudo apt-get install libexpat1-dev zlib1g-dev libboost-all-dev libssl-dev graphviz

  sudo apt-get install texlive-latex-base texlive-latex-extra texlive-pictures
  ```

- From SUNLearn, download `emdw_de424.tgz`
- Confirm the directory of the download is `Downloads` by running:

  ```
  ls /mnt/c/Users/<Your C Drive Username>/Downloads/emdw_de424.tgz
  ```

  This should return the directory coloured green.

- Navigate to your home directory in the terminal, and extract the tarball using

  ```
  tar -xvzf /mnt/c/Users/<Your C Drive Username>/Downloads/emdw_de424.tgz
  ```

  such that the following three directories exist:

  - `~/devel/gLinear`
  - `~/devel/prlite`
  - `~/devel/emdw`

- Add `~/bin` to your search path by including the following line at the bottom of your `~/.bashrc` file:

```
export PATH=$HOME/bin:$PATH
```

Your bin directory will now automatically be included in the search path for all future terminal windows you open. An existing terminal window can be updated by doing a:

```
source ~/.bashrc
```

- Symlink the necessary libraries from your `~/bin` directory

```
cd ~/bin
ln -sf $HOME/devel/gLinear/build/$(uname -s)-Release-$(uname -m)/libgLinear.so
ln -sf ~/devel/prlite/build/src/libprlite.so
ln -sf ~/devel/emdw/build/src/libemdw.so
```

Initially those symlinks will be invalid (coloured red), but don't worry. They will get fixed later.

- Compile the `gLinear` library

```
cd ~/devel/gLinear
mkdir -p build
./cmake/configure -g Release
cd build/Linux-Release-x86_64
make -j3
```

There might be some warnings, but if you see the lines

```
Built target gLinear
Built target testnew
```

everything should be fine.

- Compile the `prlite` library

```
cd ~/devel/prlite
mkdir -p build
cd build
cmake ../
make prlite -j4
```

If you see the line

```
Built target prlite
```

everything should be fine.

- Compile the `emdw` library

```
cd ~/devel/emdw
mkdir -p build
cd build
cmake ../
make emdw -j4
```

If you see the line

```
Built target emdw
```

everything should be fine.

- In Visual Studio Code, click on the blue `<>` in the bottom left corner and select `Connect to WSL`
- Next go to `File > Open Folder`
- Navigate to `/home/<username>/devel/emdw/`, click `Open` and click `Yes, I trust the authors`.
- Select the compiler kit to be the largest `GCC` if there are multiple `GCC`'s

- You could have also ran `code .` from the emdw directory in the terminal
- Test if the installation was successful by opening a terminal in Visual Studio Code and typing

  ```
  cd build; cmake ../; make -j7 example; src/bin/example; cd ..
  ```

- If you want to create a new target `week1.cc`
  - Copy `devel/emdw/src/bin/example.cc` to `devel/emdw/src/bin/week1.cc`
  - Open `devel/emdw/src/bin/CMakeLists.txt` and duplicate the bottom two lines
  - Edit these new lines by replacing `example` with `week1`
  - Edit `week1.cc` for your purpose
  - Execute the target as follows

    ```
    cd build; cmake ../; make -j7 week1; src/bin/week1; cd ..
    ```

User guide

In the `emdw` docs directory you will find a `userguide.tex` Latex (install it if you don't already have it) file. Compile it with

```
pdflatex userguide
```

It gives background information on the details of the whole emdw system – read it first.

Please follow the project's layout and logic when you expand or add any emdw features or applications. To do this properly, you will have to familiarise yourself with the `emdw/src/CMakeLists.txt` file and the subdirectory `CMakeLists.txt` files.

When extending `emdw` with new objects (such as factors and their operators etc), it is a good idea to duplicate and rename existing components. Use appropriate subdirectories and add the names of the new files to the appropriate `CMakeLists.txt` file.